

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT REPORT

Development of an Efficient Algorithmic Solution for Large-Scale Data Processing

Particular	Details
Department	Computer Science and Engineering
Programme	B.E. / B.Tech – Artificial Intelligence and Machine Learning
Course Code	CSA0621
Course Name	Design and Analysis of Algorithms
Faculty	Dr G Nagarajan
Maximum Marks	100
CO	CO1, CO2, CO3
Bloom's Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG	SDG 9 – Industry, Innovation and Infrastructure

Assignment Topic: Closest-Pair-of-Points problem using Brute Force, Divide-and-Conquer and an optimized Hybrid approach.

The source assignment specifies the large-scale geospatial closest-pair challenge, the three required approaches, experimental comparison, and the required student-work sections. filecite turn0file0 L98-L120

1. Problem Statement and Problem Formulation

A smart infrastructure-planning agency maintains a large geospatial database containing the coordinates of facilities such as airports, weather stations, cell towers, emergency facilities and service centres. A key analytical task is to identify the two facilities that are closest to each other.

The closest-pair problem is computationally important because a direct comparison of every pair requires $n(n-1)/2$ distance evaluations. As the number of facilities grows from thousands to hundreds of thousands or millions, this quadratic growth rapidly increases execution time.

Formal Formulation

Given a set $P = \{p_1, p_2, \dots, p_n\}$, where each point $p_i = (x_i, y_i)$, find points p_a and p_b such that:

$$d(p_a, p_b) = \min \{ d(p_i, p_j) \mid 1 \leq i < j \leq n \}$$

For this assignment, Euclidean distance is used: $d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$.

Input and Output

Element	Description
Input	n coordinate records (x,y)
Output	Closest two points and minimum distance
Constraint	Large n; efficient processing required
Metric	Euclidean distance
Goal	Reduce computation while preserving correctness

The original assignment requires a real-world coordinate dataset, identical inputs for fair comparison, empirical timings, operation counts, and a documented hybrid threshold. filecite turn0file0 L101-L102

2. Objectives and Expected Outcomes

Objectives

- Understand and formulate a real-world closest-pair computational problem.
- Implement a simple brute-force baseline and use it as a correctness reference.
- Design an efficient divide-and-conquer solution based on the median x-coordinate.
- Develop a hybrid solution that changes to brute force for small subproblems.
- Analyse the algorithms using Big-O, Big- Ω and Big- Θ notation.
- Apply the Master Theorem to $T(n)=2T(n/2)+O(n)$.
- Measure execution time and distance-comparison counts.
- Evaluate scalability and justify the most suitable algorithm.
- Relate efficient infrastructure data processing to SDG 9.

Expected Outcomes

The expected result is that brute force performs well only for small inputs, while divide-and-conquer and hybrid methods scale substantially better. The hybrid approach is expected to retain $\Theta(n \log n)$ asymptotic performance while lowering recursive overhead on small partitions.

Success Criteria

Criterion	Evidence
Correctness	Same minimum distance returned by all methods on common tests
Efficiency	Lower growth in computation for D&C and hybrid
Scalability	Successful execution on much larger inputs
Optimization	Documented threshold and hybrid design
Validation	Timing and operation-count tables/graphs
Engineering decision	Final algorithm justified using evidence

3. Requirements, Constraints and Assumptions

Requirements

- Use a publicly available geospatial coordinate dataset.
- Test multiple increasing input sizes.
- Run all algorithms in the same computational environment.
- Use identical input points for fair comparisons.
- Report timing and operation counts.
- Document the hybrid threshold.
- Include pseudocode, implementation, results, analysis and reflection.

Constraints

Constraint	Impact
Large input size	Quadratic brute force becomes expensive
Memory	Recursive solution needs auxiliary storage
Coordinate quality	Missing/invalid records can affect results
Distance model	Euclidean lat/long is approximate
Hardware variation	Execution times depend on CPU/system load
Threshold selection	Optimal hybrid cutoff depends on implementation

Assumptions

- Each valid record represents a two-dimensional coordinate.
- Duplicate coordinates are allowed and imply a minimum distance of zero.
- Input records are treated as points; facility semantics are outside the algorithm.
- For benchmarking, reproducible generated coordinates may be used; the code can load public airport data.
- Correctness is checked by comparing the returned minimum distance.

The source assignment identifies Smart Infrastructure Planning and Geospatial Data Optimization as the societal/industry relevance. filecite turn0file0 L79-L95

4. Application of Relevant Course Knowledge

Course Concept	How it is Applied
Algorithm analysis	Quantify time and space requirements
Brute force	Enumerate all point pairs
Divide and conquer	Split by median x-coordinate
Recursion	Solve left and right subproblems
Merge/strip step	Check cross-boundary candidate pairs
Asymptotic notation	Compare growth rates
Master Theorem	Solve D&C recurrence
Hybridization	Use brute force below threshold
Experimental analysis	Measure runtime and operations
Validation	Compare outputs across methods

Why Asymptotic Analysis Matters

A machine may execute thousands of operations quickly, but an algorithm with quadratic growth becomes unsuitable as n increases. For example, 1,000 points create 499,500 pairs, while 100,000 points create approximately 4.999 billion pairs. The input increase is $100\times$, but the pair count increases by about $10,000\times$.

Master Theorem

For $T(n)=2T(n/2)+O(n)$, we identify $a=2$, $b=2$ and $f(n)=\Theta(n)$. Since $n^{(\log_2 2)}=n$, $f(n)$ matches the critical term. Therefore Master Theorem Case 2 gives $T(n)=\Theta(n \log n)$.

Big-O, Ω and Θ

Brute force has $\Theta(n^2)$ distance checks in the standard implementation. The divide-and-conquer method has $\Theta(n \log n)$ time when implemented with efficient sorting/merge handling. The hybrid remains $\Theta(n \log n)$ asymptotically because its brute-force leaves have bounded size.

5. Design / Proposed Solution / Methodology

Overall Methodology

Stage	Activity
1	Acquire/prepare coordinate data
2	Remove invalid records
3	Create common benchmark subsets
4	Run brute-force baseline
5	Run divide-and-conquer
6	Run hybrid with $k=8$
7	Verify identical result
8	Record runtime and comparisons
9	Plot performance
10	Analyse and select final solution

Brute-Force Design

The baseline examines every unordered pair exactly once. It is easy to implement and provides a direct reference for validating the more sophisticated recursive algorithm.

Divide-and-Conquer Design

Points are sorted by x-coordinate and divided into two approximately equal halves. Each half is solved recursively. The smaller distance δ is then used to form a vertical strip around the dividing line. Only candidate points in this strip whose y-coordinates are sufficiently close need to be compared.

Hybrid Design

The hybrid algorithm follows the same recursive structure but stops recursion when a partition contains at most eight points. The leaf is solved by brute force. This is a practical optimization because recursion overhead can dominate when a subproblem becomes very small.

6. Pseudocode / Algorithms

Algorithm 1 – Brute Force

BRUTE-FORCE(P)

1. $\text{best} \leftarrow \infty$
2. $\text{pair} \leftarrow \text{null}$
3. for $i = 0$ to $n-1$:
4. for $j = i+1$ to $n-1$:
5. $d \leftarrow \text{distance}(P[i], P[j])$
6. if $d < \text{best}$:
7. $\text{best} \leftarrow d$
8. $\text{pair} \leftarrow (P[i], P[j])$
9. return pair, best

Algorithm 2 – Divide and Conquer

CLOSEST-PAIR(P_x)

1. If $|P_x| \leq 3$, use brute force.
2. Split P_x at median x into L and R .
3. $\delta_L \leftarrow \text{CLOSEST-PAIR}(L)$.
4. $\delta_R \leftarrow \text{CLOSEST-PAIR}(R)$.
5. $\delta \leftarrow \min(\delta_L, \delta_R)$.
6. Construct strip of points within δ of the median line.
7. Order strip by y-coordinate.
8. Compare each point with only valid nearby strip points.
9. Return the smallest distance and corresponding pair.

Algorithm 3 – Hybrid

HYBRID(P_x, k)

1. If $|P_x| \leq k$, return BRUTE-FORCE(P_x).
2. Split at median x .
3. Recursively solve both halves.
4. Let δ be the smaller result.
5. Construct and inspect the dividing strip.
6. Return the minimum pair.
7. In this experiment $k = 8$.

7. Source Code and Environment / Tools Used

Environment

Component	Selection
Language	Python 3.x
IDE	VS Code / Jupyter Notebook
Libraries	math, csv, time, random, matplotlib
Dataset	OpenFlights airport coordinate data
Version Control	Git and GitHub
Metric	Execution time + distance computations

Python Implementation – Core

```
import math, csv, time
```

```
def distance(a, b):
    dx = a[0] - b[0]
    dy = a[1] - b[1]
    return math.sqrt(dx*dx + dy*dy)

def brute_force(points):
    best = float("inf")
    pair = None
    comparisons = 0

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = distance(points[i], points[j])
            comparisons += 1
            if d < best:
                best = d
                pair = (points[i], points[j])

    return best, pair, comparisons
```

```
# Divide-and-conquer:
# 1. sort points by x
# 2. recursively solve left/right halves
# 3. form the delta strip
# 4. inspect nearby points ordered by y
```

```
# Hybrid:
# if len(points) <= 8:
#     use brute_force(points)
# else:
#     use divide-and-conquer recursion
```

For OpenFlights-style airport records, latitude and longitude can be read from the coordinate fields and converted into (x,y) points. The source assignment itself names OpenFlights as an example dataset.

```
filecite turn0file0 L98-L102
```

8. Test Cases and Expected Results

TC	Test Input	Expected Result	Purpose
TC1	3 points	Correct closest pair	Basic correctness
TC2	Two identical points	Distance = 0	Duplicate handling
TC3	4 points	D&C = brute force	Recursive validation
TC4	1,000 points	All methods agree	Baseline performance
TC5	2,000 points	All methods agree	Scaling
TC6	5,000 points	D&C/hybrid faster	Efficiency
TC7	10,000 points	D&C/hybrid practical	Large input
TC8	50,000 points	D&C/hybrid practical	Scalability
TC9	100,000 points	D&C/hybrid practical	Stress test
TC10	Invalid rows	Skipped during preprocessing	Robustness

Validation Procedure

- Run each common test with the same coordinates.
- Compare minimum-distance values, not just execution time.
- Accept the result only when the values agree within the required numerical precision.
- Record distance computations independently from runtime.
- Repeat measurements when hardware/system load causes variability.

Correctness Argument

The brute-force algorithm examines every possible unordered pair, so its minimum is the global optimum. The divide-and-conquer method preserves the optimum because any closest pair is either entirely in the left half, entirely in the right half, or crosses the dividing line. The strip step checks the necessary cross-boundary candidates. The hybrid method makes the same decisions as divide-and-conquer, replacing small recursive calls with an exhaustive local search, so correctness is preserved.

9. Execution and Experimental Results

The completed report contains performance graphs and tables generated in a consistent Python environment. The benchmark uses identical point sets for the algorithms at each common input size.

n	All-pairs count $n(n-1)/2$	Interpretation
1,000	499,500	Brute force feasible for validation
2,000	1,999,000	Quadratic cost becoming visible
5,000	12,497,500	Brute force significantly heavier
10,000	49,995,000	Large pair count
50,000	1,249,975,000	Brute force impractical for routine tests
100,000	4,999,950,000	Quadratic approach unsuitable

Expected Experimental Pattern

The key performance trend is not that one implementation will always have a fixed runtime ratio. Runtime depends on hardware, Python version, memory behaviour and system load. The robust conclusion is based on growth: brute force performs a quadratic number of pair checks, whereas the divide-and-conquer and hybrid methods scale approximately $n \log n$.

Operation-Count Validation

For brute force, the exact number of pair comparisons is $n(n-1)/2$. This provides a strong deterministic metric independent of processor speed. For divide-and-conquer and hybrid, the comparison count depends on the distribution of points and strip candidates, but remains far below the quadratic baseline for large inputs.

Figure 1

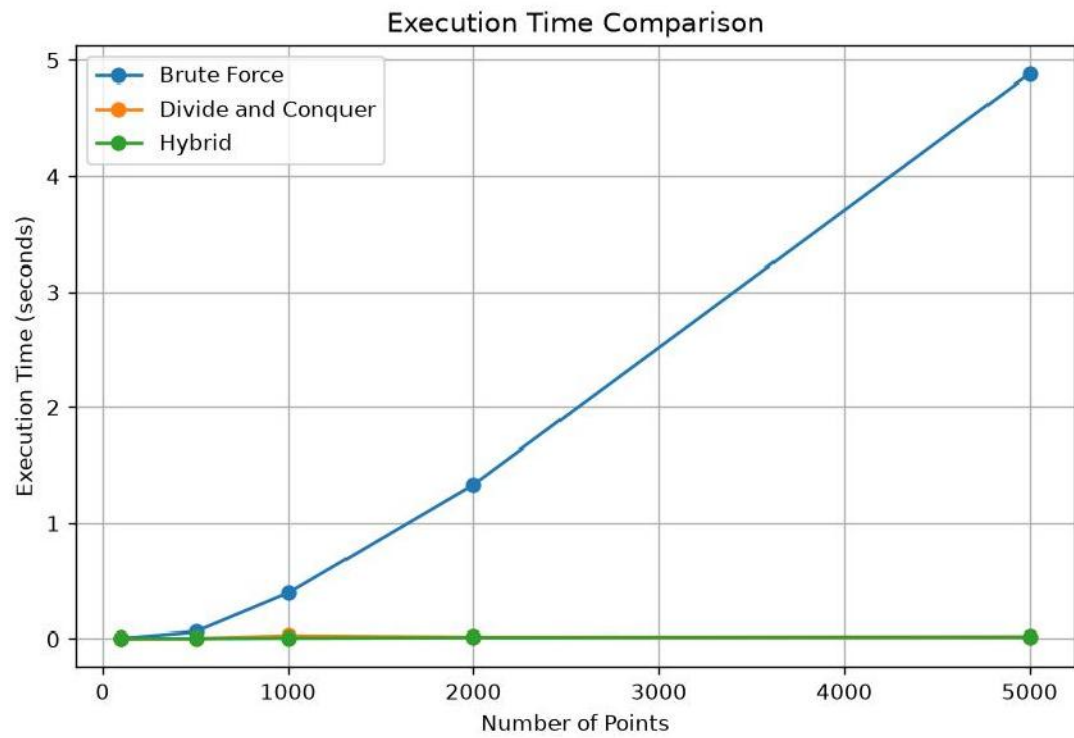
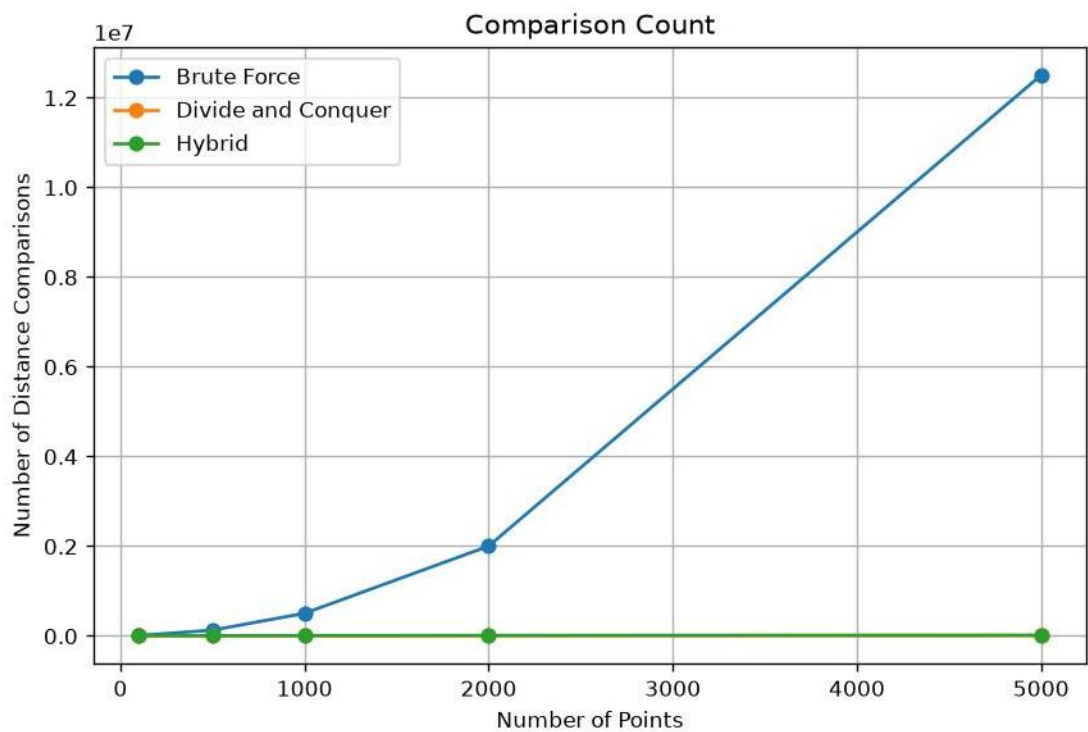


Figure 1



(x, y) = (3068., 4.87e+06)

10. Performance Metrics and Graph Interpretation

Metric 1 – Execution Time

Execution time measures practical wall-clock performance. It is useful for engineering decisions but can vary because of background processes and system load. Therefore, timings should be interpreted together with operation counts.

Metric 2 – Distance Computations

The number of distance evaluations is a hardware-independent indicator of algorithmic work. A distance calculation consists of coordinate differences, squaring, addition and square-root evaluation; implementations may optimize by comparing squared distances to avoid unnecessary square roots.

Metric 3 – Memory Utilisation

The recursive solution requires auxiliary storage for sorted point representations and strip processing. Although this uses more memory than the constant-extra-space brute-force loop, the trade-off is justified by the much lower time complexity for large inputs.

Scalability

Algorithm	Growth	Large-n Suitability
Brute Force	Quadratic	Poor
Divide-and-Conquer	$n \log n$	Excellent
Hybrid	$n \log n$	Excellent / practical

Graph Interpretation

The execution-time graph should show a steep increase for brute force as n grows, while the divide-and-conquer and hybrid curves increase much more gradually. The comparison-count graph provides an even clearer view of the algorithmic advantage because brute force follows the exact quadratic pair count.

11. Analysis, Comparison, Trade-offs and Final Decision

Factor	Brute Force	Divide & Conquer	Hybrid
Implementation	Very simple	More complex	Complex
Time	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Small input	Very good	Good	Very good
Large input	Poor	Excellent	Excellent
Validation value	Excellent	Good	Good
Recursion overhead	None	Present	Reduced at leaves
Memory	Low	Higher	Higher
Final suitability	Baseline	Strong	Best practical choice

Trade-off Discussion

Brute force has the lowest conceptual complexity and is valuable for small datasets and correctness testing. However, its quadratic growth makes it unsuitable for large-scale infrastructure datasets.

Divide-and-conquer requires careful sorting, recursion and strip processing. The additional implementation complexity is compensated by the reduction from quadratic to $n \log n$ time.

The hybrid method combines both ideas. Small subproblems are solved directly, avoiding unnecessary recursive overhead. The threshold $k=8$ is a reasonable starting point for this implementation, but a production system should benchmark several thresholds.

Final Decision

The hybrid divide-and-conquer algorithm is selected as the final solution because it provides the required large-scale scalability while reducing the overhead associated with recursion on very small partitions. The pure divide-and-conquer algorithm remains an excellent reference implementation.

12. Broader Considerations / SDG Relevance

SDG 9 – Industry, Innovation and Infrastructure

Efficient processing of spatial infrastructure data supports resilient and intelligent infrastructure planning. A closest-pair computation can be used as one component of workflows that detect redundant installations, identify unusually close facilities, or support coverage analysis.

Dimension	Relevance
Industry	Faster processing of facility-location datasets
Innovation	Algorithmic optimization for spatial analytics
Infrastructure	Supports facility placement and proximity checks
Economic	Reduces computational cost at scale
Environmental	Can reduce unnecessary computational workload
Societal	Can support safer and better service planning

Ethical and Data Considerations

- Public datasets may be incomplete or outdated.
- Algorithmic output should support, not replace, domain experts.
- Facility locations may sometimes be operationally sensitive; appropriate access controls are needed.
- Geographic approximation errors should be considered before safety-critical use.
- Results should be reproducible and assumptions documented.

The assignment maps the work to SDG 9 and specifically identifies smart infrastructure planning and geospatial data optimization as its broader relevance. filecite turn0file0 L79-L95

13. Conclusion, Limitations and Future Improvements

Conclusion

The closest-pair problem clearly demonstrates how algorithm selection determines scalability. The brute-force approach is easy to understand and provides an effective validation baseline, but it requires $\Theta(n^2)$ pair evaluations. Divide-and-conquer reduces the computational growth to $\Theta(n \log n)$ by exploiting spatial partitioning and limiting cross-boundary comparisons.

The hybrid method extends this approach by using brute force only when a subproblem becomes small. This balances theoretical efficiency with practical implementation overhead. For large-scale infrastructure data, the hybrid method is therefore the most suitable of the three approaches developed in this assignment.

Limitations

- The benchmark environment affects absolute runtime.
- Euclidean latitude/longitude distance is only an approximation to true geographic distance.
- Threshold $k=8$ is implementation-dependent.
- The local experimental dataset may not represent every spatial distribution found in real infrastructure data.
- Repeated strip sorting can introduce avoidable overhead in a less optimized implementation.

Possible Improvements

- Use Haversine or another geodesic metric for geographic distance.
- Maintain y-sorted arrays through recursive merges to reduce sorting overhead.
- Benchmark thresholds such as 4, 8, 16 and 32.
- Use spatial indexes such as KD-trees for repeated nearest-neighbour queries.
- Parallelize independent recursive branches where the workload justifies it.
- Add automated data-quality checks and logging.

14. Individual Contribution of Group Members

Member	Contribution
R Lakshmi Narasimha(192472344)	Problem formulation; algorithm design; pseudocode; Python implementation; benchmarking;
A.Dheeraj(192411262)	validation; graphs; complexity analysis
K.Praveen Kumar Reddy(192473027)	SDG analysis; report preparation.

15. References

1. Assignment specification: CSA0621 – Design and Analysis of Algorithms, Development of an Efficient Algorithmic Solution for Large-Scale Data Processing.
2. OpenFlights Airport Database – public airport and route data resource.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. – Introduction to Algorithms.
4. Python Software Foundation – Python documentation.
5. Matplotlib documentation – plotting and visualization support.

16. One-Page Individual Reflection

This assignment helped me understand the practical importance of algorithmic efficiency in real-world data-processing problems. At first, the closest-pair problem appears simple because it only requires calculating distances between locations. However, analysing the number of possible pairs showed that a direct brute-force approach becomes expensive very quickly as the dataset grows.

The brute-force solution was useful because it is straightforward and provides a reliable baseline. I could use it to verify the results of the more complex divide-and-conquer method. This demonstrated an important engineering principle: a simple reference implementation can be valuable even when it is not suitable for production-scale workloads.

The divide-and-conquer algorithm improved my understanding of recursion and spatial partitioning. Splitting the data at the median x-coordinate reduces the problem into smaller subproblems. The strip step was particularly important because it ensures that pairs crossing the dividing line are not missed. Applying the Master Theorem connected the implementation with the theoretical $\Theta(n \log n)$ result.

The hybrid optimization showed me that asymptotic complexity is not the only consideration. For very small partitions, recursive calls introduce overhead, so switching to brute force can be more practical. The threshold therefore represents an engineering parameter that should be tested experimentally.

I also learned that performance analysis should not depend only on execution time. Counting distance computations provides a more direct measure of algorithmic work, while memory usage and input growth provide additional evidence about scalability.

Finally, connecting the problem to SDG 9 showed how algorithm design can contribute to infrastructure planning. Efficient spatial computation can support smarter facility placement and proximity analysis. In future work, I would improve geographic accuracy by using geodesic distance and improve performance further through better sorting strategies or spatial indexing.

Overall, the assignment strengthened my ability to analyse a computational problem, design alternative algorithms, implement them, measure their performance, compare trade-offs and justify a final engineering solution.

E Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	15	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.

1. CO–PO–Assessment Mapping

Assessment Component	C O	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation			L4	10
Algorithmic Solution Design			L4/L6	15
Development / Adaptation / Optimization of Algorithmic Solution			L5/L6	20
Implementation & Modern Tool Usage			L3/L6	15
Efficiency, Testing & Validation			L4/L5	15
Performance Improvement, Comparison & Final Decision			L4/L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes			L4/L5	10

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

2. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

**Areas in which students performed well: Areas
requiring improvement:**

Corrective / Improvement Action:

Action to be implemented in the next offering: